



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Sanchez Villalpando Johan

N° de Cuenta: 422028657

GRUPO DE LABORATORIO: 04

GRUPO DE TEORÍA: 02

SEMESTRE 2025-2

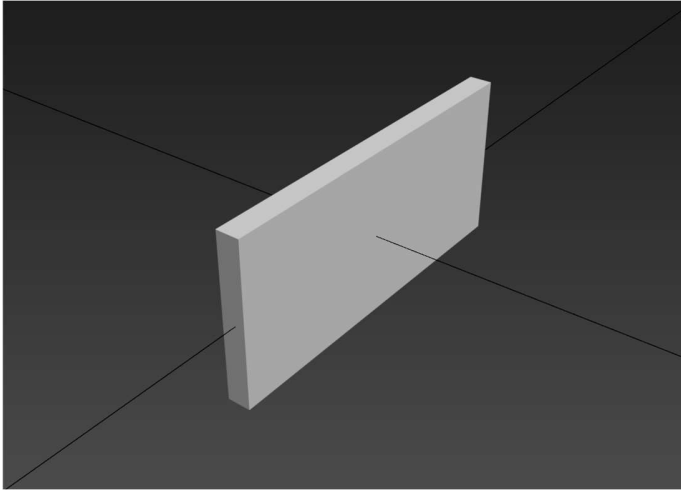
FECHA DE ENTREGA LÍMITE: 26/04/25

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Para el primer ejercicio se requiere separar el cartel de la puerta con arco realizada en el previo, para ello se hizo uso de 3dsmax.



Una vez separado se añadió al código junto a la puerta

```
Model puerta_M;  
Model letrero_M;
```

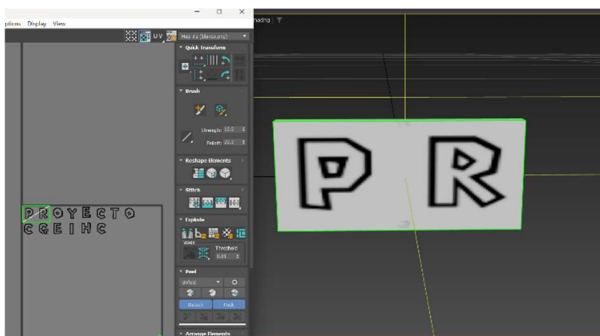
```
//puerta  
puerta_M = Model();  
puerta_M.LoadModel("Models/PUERTA_ARCO.obj");  
  
letrero_M = Model();  
letrero_M.LoadModel("Models/letrero.obj");
```

Para el segundo ejercicio se requiere animar la palabra PROYECTO CGEIHC usando la fuente del previo, en el cartel de manera cíclica, como un letrero LCD.

Para la animación se creó una imagen nueva, la cual tendrá como textura el cartel, esta imagen fue diseñada de una manera específica para poderse animar, usando un editor de imágenes y la imagen de la fuente utilizada en el previo.

PROYECTO
ENGINE

Esta imagen se puso como textura en el letrero, con un zoom para poder apreciar las letras de proyecto.



De este modo dentro del código se usará un offset para moverse a través de la textura.

Se instancia la puerta

```
//instancia de la puerta
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(4.0f, -2.0f, 4.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
puerta_M.RenderModel();
```

Posteriormente se crea la animación del cartel.

Para ello se crean variables `toffsetletrero` y `toffsetletrero`, a base de condicionales se realiza la animación, iniciando siempre con un movimiento aumentando en `u` para recorrer la primera palabra (0.8 en `u`), una vez se ha recorrido, se revisa que no se haya recorrido la segunda palabra, en caso de que no (`toffsetletrero`=0) se baja un poco para pasar a la siguiente palabra, una vez la recorre se hace un salto sin reiniciar el recorrido en `u`, de modo que se muestre una parte blanca antes de PROYECTO, para dar la ilusión de que proyecto viene de manera cíclica.

```

//instancia del letrero
//textura con movimiento
if (toffsetletrrou < 0.81) { //hasta terminar proyecto
    toffsetletrrou += 0.005 * deltaTime; //aumenta en u
}
else if (toffsetletrrou >= 0.81 && toffsetletrrov == 0) { //empieza cgeihc
    toffsetletrrov = 0.9; //baja en v para ver las letras del segundo párrafo
    toffsetletrrou = 0.0; //reinicia u para mostrar palabra completa
}
else if (toffsetletrrou >= 0.81 && toffsetletrrov > 0.0) { //sube en v para mostrar proyecto desde el inicio
    toffsetletrrov = 0.00001; //sube en v para ver las letras del primerr párrafo
    toffsetletrrou += 0.005 * deltaTime; //aumenta en u

    if (toffsetletrrou > 1.0) { //para evitar desborde
        toffsetletrrou = 0.0;
        toffsetletrrov = 0.0; //sube y reinicia
    }
}

//printf("\n toffsetletrrou: %f", toffsetletrrou);
//printf("\n toffsetletrrov: %f", toffsetletrrov);

```

Se instancia el letrero y se le pasan los valores de toffsetletrero

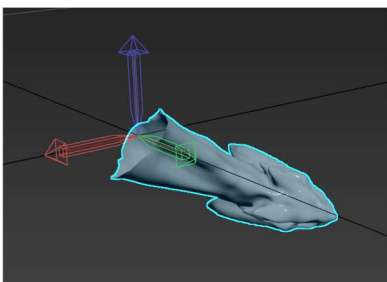
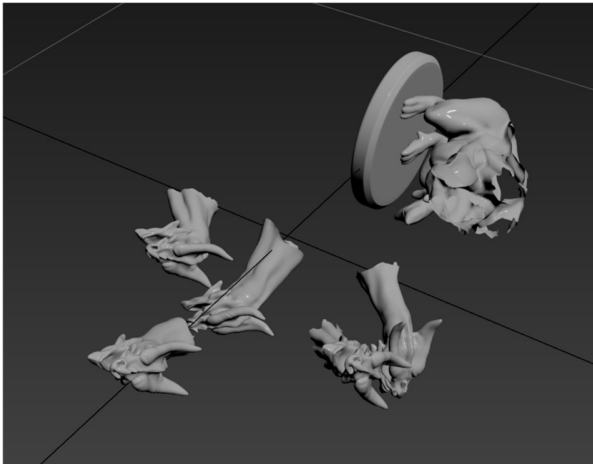
```

toffset = glm::vec2(toffsetletrrou, toffsetletrrov);
model = glm::translate(model, glm::vec3(0.0f, 4.3f, 0.0f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
letrero_M.RenderModel();

```

Para el tercer ejercicio se tienen que separar las cabezas del dragón y animarlas independientemente, además de asignarles un color.

Para ello en 3dsmax se separaron las cabezas y se exportó cada una independientemente, además del nuevo cuerpo, todas con su respectivo ajuste de pivote.



Posteriormente se importan en el proyecto.

```
//cabeas
Model cabeza1;
Model cabeza2;
Model cabeza3;
Model cabeza4;
Model cabeza5;
//ALAS
Model alaup_M;
Model aladown_M;

//cabezas
cabeza1.LoadModel("Models/cabeza1.obj");
cabeza2.LoadModel("Models/cabeza2.obj");
cabeza3.LoadModel("Models/cabeza3.obj");
cabeza4.LoadModel("Models/cabeza4.obj");
cabeza5.LoadModel("Models/cabeza5.obj");

//ALAS
alaup_M = Model();
alaup_M.LoadModel("Models/alaup.obj");
aladown_M = Model();
aladown_M.LoadModel("Models/aladown.obj");
```

Para el movimiento de ida y regreso del dragón dentro de la traslación donde también realiza movimiento senoidal, añadí una función senoidal la cual funciona para ir y regresar, la distancia es de 6.0 y la velocidad es de 0.003.

De este modo como en la función senoidal, el dragón va y regresa.

```
model = glm::translate(model, glm::vec3(0.0f - sin(angulovaria*0.003) * 6.0f, 5.0f + sin(glm::radians(angulovaria)), 6.0));

model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Dragon_M.RenderModel();
modelaux2 = model;
```

Para las cabezas se añadieron y se posicionaron anclados al cuerpo.

Para la primera cabeza roja se usó una función seno sencilla con rotación en X, de modo que la cabeza realiza una animación de "Si"

```
//cabezas
//roja 1 asiente con la cabeza
model = glm::translate(model, glm::vec3(-1.8f, 0.8f, 3.5f));
model = glm::rotate(model, -0.5f * sin(angulovaria * 0.1f), glm::vec3(-1.0f, 0.0f, 0.0f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza1.RenderModel();
model = modelaux2;
```

Para la segunda cabeza verde se añadió un giro sencillo, de modo que la cabeza gira sobre su propio eje como en el exorcista.

```
//verde 2 giro de cabeza sobre si misma
model = glm::translate(model, glm::vec3(-1.7f, -0.2f, 3.3f));
model = glm::rotate(model, -0.5f * (angulovaria * 0.1f), glm::vec3(0.0f, 0.0f, 1.0f));
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza2.RenderModel();
model = modelaux2;
```

Para la tercera cabeza se usó una onda envolvente, de modo que la cabeza hace una animación de “Si” con un rebote, empieza lento, después se acelera y termina lento.

```
//blanco 3 onda envolvente
model = glm::translate(model, glm::vec3(-2.0f, 0.85f, 2.4f));
model = glm::rotate(model, -0.5f * sin(angulovaria*0.1f) * sin(0.2f * (angulovaria * 0.1f)), glm::vec3(-1.0f, 0.0f, 0.0f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza3.RenderModel();
model = modelaux2;
```

Para la cuarta cabeza azul, se añade una rotación de seno en x,z de modo que la cabeza hace un movimiento similar a negar, de izquierda a derecha.

```
//azul 4 seno con 2 rotaciones niega con la cabeza
model = glm::translate(model, glm::vec3(-2.385f, 0.26f, 2.8f));
model = glm::rotate(model, -0.5f * sin(angulovaria * 0.1f), glm::vec3(-1.0f, 0.0f, 1.0f));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza4.RenderModel();
model = modelaux2;
```

Para la quinta cabeza se utiliza una onda pulsante seno, de modo que la animación va como si se atorara en momentos, simulando latidos.

```
//cafe 5 onda pulsante
model = glm::translate(model, glm::vec3(-2.0f, -0.3f, 2.4f));
model = glm::rotate(model, -0.5f * pow(sin(angulovaria * 0.1f), 3.0f), glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(0.5f, 0.25f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cabeza5.RenderModel();
model = modelaux2;
```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Para esta práctica no hubo problemas, más que tener creatividad para buscar funciones que realicen movimientos distintos.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Considero que esta práctica fue interesante ya que descubrí como se hacen muchas animaciones en base a funciones trigonométricas y matemáticas en general, lo que a su vez me da ideas para utilizar en el proyecto.

Con respecto a la dificultad considero que no fue muy complicado y además fue bastante intuitivo, lo único tedioso fue separar las cabezas del dragón y generar la imagen para animarla, de ahí en fuera considero todo fue bien, además la explicación también me pareció adecuada.

Bibliografía en formato APA

- *Rational developer for i.* (2021, marzo 4). Ibm.com. Recuperado el 25 de abril de 2025, de <https://www.ibm.com/docs/en/rdfi/9.6.0?topic=functions-pow-compute-power>
- Jsgarcia. (2019, noviembre 12). *Modulación de amplitud (AM) y sus variantes - Modulación de amplitud (AM) y sus variantes - Medium*. Modulación de Amplitud (AM) y Sus Variantes. Recuperado el 25 de abril de 2025, de <https://medium.com/modulaci%C3%B3n-por-amplitud-am-y-sus-variantes/modulaci%C3%B3n-de-amplitud-am-y-sus-variantes-6b7d575d2698>
- *Ondas en Electrónica y Comunicaciones*. (2022, octubre 31). Electroagenda. Recuperado el 25 de abril de 2025, de <https://electroagenda.com/es/ondas-en-electronica-y-comunicaciones/>
- (S/f-b). Fontmeme.com. Recuperado el 25 de abril de 2025, de <https://fontmeme.com/fonts/static/12915/super-mario-bros-font-character-map.png>